

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 03 -

PADRÕES DE PROJETO EM ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	16

EMSE

O QUE VEM POR AÍ?

Modelos bem treinados não são suficientes quando o desafio é transformar Machine Learning em soluções reais e escaláveis. Nesta aula, você vai entender como Design Patterns ajudam a organizar, estruturar e dar inteligência ao código que sustenta sistemas de ML.

Ao longo do conteúdo, exploraremos padrões Criacionais, Estruturais e Comportamentais, conectando conceitos clássicos da engenharia de software a problemas práticos enfrentados em pipelines de dados, treinamento e produção de modelos.

Com exemplos em Python e uma abordagem aplicada, o objetivo é mostrar como escrever código mais limpo, modular e fácil de evoluir, preparando você para atuar em projetos profissionais de Machine Learning.

HANDS ON

Os Design Patterns (Padrões de Projeto) constituem soluções consolidadas para problemas recorrentes no desenvolvimento de software e desempenham papel essencial na construção de sistemas de Machine Learning mais organizados, escaláveis e sustentáveis. Em aplicações práticas, os principais desafios não se concentram apenas na modelagem estatística, mas na forma como dados, algoritmos e processos são estruturados para permitir evolução contínua, manutenção eficiente e colaboração entre equipes.

A origem dos Design Patterns está ligada à engenharia de software, como resposta à necessidade de lidar com sistemas cada vez mais complexos. Com o avanço das aplicações orientadas a dados, esses padrões passaram a ser amplamente aplicados em projetos de Machine Learning, nos quais pipelines costumam integrar múltiplas etapas, ferramentas e decisões técnicas. Sua adoção contribui para maior previsibilidade do comportamento do sistema e para a padronização das estruturas de desenvolvimento.

Os padrões de projeto são tradicionalmente organizados em três categorias: Criacionais, Estruturais e Comportamentais. Essa classificação facilita a compreensão dos diferentes tipos de problemas que surgem ao longo do desenvolvimento de sistemas de ML.

No contexto de Machine Learning, os Padrões Criacionais favorecem a construção de pipelines flexíveis, permitindo a substituição organizada de modelos, métricas e estratégias. Os Padrões Estruturais auxiliam na integração de componentes heterogêneos e no encapsulamento de complexidades internas. Os Padrões Comportamentais oferecem mecanismos para lidar com comportamentos dinâmicos, como variação de estratégias e organização de responsabilidades.

A aplicação consistente desses padrões resulta em código mais limpo, modular e reutilizável, facilitando a manutenção, a colaboração entre equipes e a evolução das soluções de Machine Learning em ambientes profissionais.

SAIBA MAIS

À medida que sistemas de Machine Learning deixam de ser apenas experimentos isolados e passam a fazer parte de produtos reais, surge um novo desafio: como estruturar o código de forma organizada, reutilizável e fácil de evoluir ao longo do tempo? Não basta que um modelo tenha boa performance estatística; ele também precisa estar inserido em um sistema bem projetado, que possa ser mantido, testado e escalado por diferentes pessoas e equipes.

Neste contexto, os Design Patterns (Padrões de Projeto) surgem como uma ponte entre a engenharia de software tradicional e o desenvolvimento de sistemas baseados em dados. Eles oferecem soluções consagradas para problemas recorrentes de projeto, ajudando a organizar responsabilidades, reduzir acoplamento e tornar o código mais legível e extensível — características fundamentais em pipelines de Machine Learning, que frequentemente envolvem pré-processamento, treinamento, validação, versionamento e deploy de modelos.



Figura 1 - Meme utilizado para design patterns
Fonte: Google Imagens (2026)

Design Patterns são descrições reutilizáveis de soluções para problemas recorrentes no projeto de software. Eles não são códigos prontos, mas sim modelos de pensamento e organização, que orientam como classes e objetos podem colaborar entre si para resolver determinado problema de forma elegante e sustentável.

Em sistemas de Machine Learning, sua importância é ainda maior. Pipelines mal estruturados tendem a crescer de forma desorganizada, com código duplicado, forte dependência entre componentes e dificuldade de adaptação a novas demandas

— como troca de modelos, inclusão de novas métricas ou mudanças na forma de validação. Os padrões de projeto ajudam a minimizar esses problemas, promovendo modularidade, clareza e facilidade de manutenção.

Do ponto de vista histórico, os Design Patterns ganharam popularidade na década de 1990 com o trabalho do grupo conhecido como Gang of Four (GoF), que sistematizou e documentou padrões amplamente utilizados na engenharia de software orientada a objetos. Desde então, esses padrões evoluíram e passaram a ser aplicados em diferentes contextos, incluindo arquiteturas modernas, sistemas distribuídos e, mais recentemente, aplicações de Machine Learning e Data Science.

Esse trabalho foi pioneiro ao organizar padrões de projeto em tipos e categorias bem definidos. A ideia central era identificar soluções reutilizáveis para problemas recorrentes no design de software, ajudando programadores(as) a evitarem reinventar a roda. A partir dessa base, os padrões de projeto começaram a ser aplicados em diversas áreas da engenharia de software, incluindo sistemas orientados a objetos, arquitetura de sistemas e, mais recentemente, em áreas como Machine Learning e Big Data.

Com a evolução da tecnologia, surgiram diferentes variantes dos padrões e muitos deles começaram a ser aplicados de maneira prática em sistemas de ML. Hoje, entender esses padrões é crucial para a criação de sistemas robustos, escaláveis e fáceis de manter, especialmente quando lidamos com dados, modelos e pipelines que podem crescer e mudar rapidamente.

Categorias de padrões de projeto

Os Design Patterns podem ser classificados em três grandes categorias, cada uma com um foco distinto e um papel essencial no design de sistemas. Vamos analisar essas categorias e como elas se aplicam aos sistemas de Machine Learning:

Os padrões criacionais são focados na criação de objetos e na gestão de suas instâncias de forma flexível e eficiente. Eles ajudam a desacoplar o código do processo de criação de objetos, permitindo uma maior flexibilidade, escalabilidade e reusabilidade.

- Exemplo em Machine Learning: o padrão Factory Method pode ser utilizado para criar instâncias de modelos de ML de acordo com a tarefa que precisa ser resolvida (por exemplo, classificação, regressão etc.). Em vez de criar manualmente cada modelo, a fábrica pode escolher e instanciar o modelo correto com base na entrada fornecida, tornando o código mais modular e fácil de manter.
- Padrões Criacionais Comuns: Singleton, Factory, Abstract Factory, Builder, Prototype.

Já os padrões estruturais lidam com a organização e a composição de classes e objetos. Eles ajudam a estruturar o código de forma que componentes diferentes possam interagir de maneira eficaz, sem criar dependências desnecessárias ou complexidade excessiva.

- Exemplo em Machine Learning: o padrão Adapter pode ser utilizado para conectar diferentes tipos de transformadores ou modelos em um pipeline de ML, mesmo que eles tenham interfaces diferentes. Isso facilita a integração de novos componentes no sistema sem precisar alterar os componentes existentes.
- Padrões Estruturais Comuns: Adapter, Composite, Decorator, Facade, Proxy, Bridge, Flyweight.

Por fim, os padrões comportamentais estão preocupados com como as classes e objetos interagem entre si. Eles descrevem maneiras de gerenciar fluxos de controle, comportamentos e responsabilidades dentro de sistemas complexos.

- Exemplo em Machine Learning: o padrão Strategy pode ser usado para permitir a troca de algoritmos de pré-processamento ou validação de modelo de forma dinâmica. Com isso, é possível mudar o comportamento do sistema sem modificar seu código principal, permitindo uma configuração mais flexível para diferentes cenários de ML.
- Padrões Comportamentais Comuns: Observer, Strategy, Command, State, Chain of Responsibility, Iterator, Mediator, Memento, Template Method.

Até aqui, tivemos uma visão geral sobre o que são os padrões de projeto, seu contexto histórico e como eles se organizam nas categorias Criacionais, Estruturais e

Comportamentais. No entanto, compreender essas categorias de forma conceitual é apenas o primeiro passo. Para que esse conhecimento se torne realmente útil no dia a dia de projetos de Machine Learning, é fundamental entender como cada padrão funciona na prática, quais problemas ele resolve e em que situações faz sentido aplicá-lo.

A partir deste ponto, a aula passa a aprofundar cada um dos padrões selecionados, explorando sua motivação, estrutura e implementação em Python, sempre conectando o conceito de engenharia de software a cenários reais encontrados em pipelines e sistemas de ML.

Padrões Criacionais

Os Padrões Criacionais são um conjunto de Design Patterns cujo foco principal está na forma como objetos são criados dentro de um sistema. Em vez de instanciar objetos diretamente espalhados pelo código, esses padrões propõem abstrair, controlar ou padronizar o processo de criação, tornando o sistema mais flexível, reutilizável e fácil de manter.

Em projetos de Machine Learning, esse cuidado é especialmente importante. É muito comum termos diferentes tipos de modelos (regressão, classificação, clustering); múltiplas estratégias de pré-processamento; configurações globais de experimento; e regras específicas para inicialização de pipelines.

Quando a criação desses objetos é feita de forma desorganizada, o código rapidamente se torna rígido, difícil de testar e complexo de evoluir. Os padrões criacionais surgem exatamente para resolver esse problema estrutural.

Em sistemas de ML, o código raramente é estático. Modelos mudam, features evoluem, métricas são ajustadas e novos experimentos surgem constantemente. Os padrões criacionais ajudam a:

- Reduzir acoplamento entre partes do sistema.
- Centralizar regras de criação de objetos complexos.
- Facilitar troca de modelos ou estratégias sem alterar o código principal.
- Tornar o projeto mais testável e escalável.

- Evitar duplicação de código e lógica espalhada.

Em termos práticos: eles ajudam a separar “o que o sistema faz” de “como os objetos são criados”. Os padrões criacionais clássicos incluem Singleton, Factory Method, Abstract Factory, Builder e Prototype.

Nesta primeira parte da disciplina, vamos focar nos mais comuns e úteis em Machine Learning, começando pelo Factory Method, por ser um dos mais aplicáveis no dia a dia de um(a) cientista de dados.

O Factory Method é um padrão criacional que define uma interface para criar objetos, mas permite que subclasses ou métodos especializados decidam qual classe concreta será instanciada.

Em vez de criar objetos diretamente usando if/else espalhados pelo código, a criação é centralizada em uma “fábrica”. O Factory Method é amplamente utilizado para:

- Instanciar diferentes modelos (sklearn, XGBoost, LightGBM, etc.).
- Criar pipelines completos de pré-processamento.
- Selecionar métricas de avaliação dinamicamente.
- Construir estratégias de validação (K-Fold, Stratified, Time Series).

Use padrões criacionais quando a criação de objetos começa a ficar repetitiva; o sistema precisa ser flexível a mudanças; existem várias variações de um mesmo tipo de objeto; ou você quer desacoplar regras de negócio da criação de instâncias.

No entanto, evite aplicar padrões criacionais em scripts pequenos ou experimentos rápidos; quando a criação de objetos é trivial; ou se o padrão adicionar mais complexidade do que benefício.

Padrões são ferramentas — não regras obrigatórias.

Em projetos profissionais de ML, padrões criacionais aparecem frequentemente em frameworks internos de experimentação, sistemas de treino automatizado APIs de inferência e plataformas de MLOps. Eles permitem que cientistas de dados foquem no modelo e nos dados, enquanto a arquitetura do sistema permanece organizada e sustentável.

Padrões Estruturais

Os Padrões Estruturais são Design Patterns que têm como foco a forma como classes e objetos são organizados e combinados para formar estruturas maiores e mais flexíveis. Enquanto os padrões criacionais tratam de como os objetos nascem, os padrões estruturais lidam com como esses objetos se conectam e colaboram entre si.

Em sistemas de Machine Learning, essa organização é essencial. Pipelines de ML geralmente envolvem múltiplos componentes — leitura de dados, pré-processamento, engenharia de atributos, modelo, avaliação e inferência. Sem uma boa estrutura, o sistema rapidamente se torna difícil de entender, modificar e reutilizar.

Os padrões estruturais ajudam justamente a reduzir o acoplamento, facilitar integrações e esconder complexidades internas, tornando o código mais limpo e sustentável.

Projetos de ML raramente trabalham com um único componente isolado. É comum lidar com bibliotecas diferentes (scikit-learn, PyTorch, TensorFlow), APIs externas, pipelines heterogêneos e sistemas legados.

Os padrões estruturais permitem integrar componentes incompatíveis, encapsular complexidade técnica, organizar pipelines extensos de forma clara, tornar sistemas mais fáceis de manter e evoluir e facilitar testes e reutilização de código. Em outras palavras, eles ajudam a dar forma ao sistema.

Os padrões estruturais clássicos incluem Adapter; Facade; Decorator; Composite; Proxy; Bridge; e Flyweight.

O Adapter é um padrão estrutural que permite que interfaces incompatíveis trabalhem juntas. Ele atua como um “tradutor”, adaptando uma interface existente para outra esperada pelo sistema. Esse padrão é extremamente útil em ML, pois frequentemente precisamos integrar bibliotecas ou componentes que não seguem o mesmo padrão de interface.

Imagine um pipeline que espera um método `fit()` e `predict()`, mas você quer usar um modelo externo com uma interface diferente. Assim, ele aparece em ML na integração entre diferentes frameworks; padronização de modelos customizados;

conexão entre código legado e pipelines modernos; e unificação de interfaces em sistemas de experimentação.

Já Facade é um padrão estrutural que fornece uma interface simplificada para um sistema complexo. Ele esconde detalhes internos e oferece ao usuário uma forma fácil de interagir com o sistema. Em ML, o Facade é muito comum quando queremos encapsular um pipeline inteiro atrás de uma interface simples.

Assim, o Facade é usado em ML em pipelines de treino e inferência, APIs de modelos, sistemas de MLOps e frameworks internos de dados.

Use padrões estruturais quando o sistema começa a ficar difícil de entender, existem muitas dependências entre componentes, você precisa integrar ferramentas diferentes ou quer esconder complexidade do usuário final.

Evite padrões estruturais quando o sistema ainda é pequeno, a estrutura é simples e clara, o padrão adiciona camadas desnecessárias ou estrutura demais também pode atrapalhar.

Em projetos profissionais de ML, padrões estruturais aparecem em pipelines de dados, sistemas de inferência, integrações com APIs e plataformas internas de ciência de dados. Eles permitem que sistemas cresçam sem virar um emaranhado de código.

Padrões Comportamentais

Os Padrões Comportamentais são Design Patterns focados em como os objetos se comunicam entre si e como responsabilidades e comportamentos são distribuídos dentro do sistema. Diferente dos padrões criacionais (criação) e estruturais (organização), aqui o foco está na dinâmica do sistema em execução.

Em Machine Learning, essa categoria é extremamente relevante, pois muitos sistemas precisam alternar algoritmos dinamicamente, reagir a eventos (treino finalizado, modelo atualizado), encapsular regras de decisão ou controlar fluxos complexos de execução. Os padrões comportamentais ajudam a organizar essas interações, evitando lógica condicional excessiva (if/else) e código difícil de manter.

Projetos de ML não são lineares. Eles envolvem decisões como qual estratégia de pré-processamento usar? qual métrica avaliar? qual método de validação aplicar?

quando disparar um retreino? Sem padrões comportamentais, essas decisões acabam espalhadas pelo código, tornando o sistema difícil de entender, frágil a mudanças, pouco reutilizável e complicado de testar.

Os padrões comportamentais permitem variar o comportamento do sistema sem alterar sua estrutura, o que é essencial em ambientes experimentais e produtivos de ML. Os padrões comportamentais clássicos incluem Strategy; Observer; Command; State; Chain of Responsibility; Template Method; Iterator; Mediator.

O Strategy é um padrão comportamental que define uma família de algoritmos, encapsula cada um deles e permite que eles sejam intercambiáveis em tempo de execução. Em vez de fixar um comportamento no código, o Strategy permite escolher qual algoritmo ou regra será aplicada, sem alterar a lógica principal do sistema.

O Strategy aparece em ML em escolha de algoritmos de pré-processamento, métodos de validação, estratégias de balanceamento de dados, funções de custo customizadas e políticas de decisão em produção.

O Observer é um padrão comportamental que define uma relação de dependência um-para-muitos entre objetos. Quando o objeto principal muda de estado, todos os seus observadores são notificados automaticamente. Em ML, esse padrão é muito útil para reagir a eventos.

Exemplos de uso em ML incluem treinamento finalizado, novo modelo versionado, performance abaixo do esperado ou se o pipeline falhou. Ele é usado em logging de eventos, monitoramento de performance, alertas de produção, gatilhos de retreino automático e integração com sistemas de MLOps.

Use padrões comportamentais quando há muitos if/else controlando comportamento, o sistema precisa reagir a eventos, algoritmos precisam ser intercambiáveis ou o fluxo de execução é complexo.

Evite padrões comportamentais quando o comportamento é simples e fixo, o padrão adiciona abstrações desnecessárias ou, o sistema ainda está em fase exploratória. Abstração demais pode atrapalhar a clareza.

Em projetos reais de ML, padrões comportamentais são usados em frameworks de experimentação, sistemas de validação automática, monitoramento de modelos,

pipelines de retreinamento e plataformas de decisão baseadas em modelos. Eles permitem que o sistema se adapte a mudanças sem ser reescrito.

Ao longo desta aula, ficou evidente que os Design Patterns não são apenas conceitos teóricos da engenharia de software, mas sim ferramentas práticas e fundamentais para a construção de sistemas de Machine Learning mais robustos e profissionais. Em projetos reais, os desafios raramente estão apenas na escolha do melhor modelo, mas na capacidade de escalar soluções, manter o código compreensível ao longo do tempo e adaptar o sistema a novas demandas sem reescrevê-lo por completo. É justamente nesse cenário que os padrões de projeto se tornam essenciais, pois oferecem soluções consolidadas para problemas recorrentes de escalabilidade, modularidade e manutenção.

Ao aplicar padrões criacionais, estruturais e comportamentais, o(a) desenvolvedor(a) passa a separar responsabilidades, reduzir acoplamentos desnecessários e organizar melhor os fluxos de decisão e interação entre componentes. Isso resulta em código mais limpo, previsível e testável, características indispensáveis em pipelines de Machine Learning que evoluem continuamente. Além disso, sistemas bem projetados facilitam a troca de modelos, a inclusão de novas estratégias e a integração com ferramentas de MLOps, sem comprometer a estabilidade da aplicação.

Outro ponto central é o impacto dos Design Patterns na colaboração em projetos grandes e multidisciplinares. Em equipes de dados, em que indivíduos cientistas de dados, engenheiros de ML e engenheiros de software atuam em conjunto, padrões de projeto criam uma linguagem comum de design, tornando o código mais fácil de entender, revisar e estender. Dessa forma, o conhecimento deixa de ficar concentrado em indivíduos e passa a ser compartilhado pela própria estrutura do sistema, aumentando a qualidade, a sustentabilidade e a vida útil das soluções de Machine Learning desenvolvidas.

O QUE VOCÊ VIU NESTA AULA?

Os Design Patterns (Padrões de Projeto) representam um conjunto de soluções consolidadas para problemas recorrentes no desenvolvimento de software e assumem papel estratégico na construção de sistemas de Machine Learning robustos, escaláveis e sustentáveis. Em contextos profissionais, a qualidade de uma solução de ML não é definida apenas pela performance do modelo, mas pela capacidade do sistema de evoluir, integrar novos componentes e manter clareza estrutural ao longo do tempo. Nesse cenário, os padrões de projeto oferecem diretrizes fundamentais para organizar código, reduzir complexidade e aumentar a confiabilidade das aplicações.

Os padrões de projeto são tradicionalmente organizados em três grandes categorias: Criacionais, Estruturais e Comportamentais.

Os Padrões Criacionais concentram-se no controle da criação de objetos e na centralização de regras de instância e configuração. Em sistemas de Machine Learning, esses padrões são especialmente úteis para lidar com a diversidade de modelos, métricas, pipelines e estratégias que podem variar ao longo do ciclo de vida do projeto. Ao aplicar padrões criacionais, torna-se possível reduzir o acoplamento entre componentes, facilitar a substituição de algoritmos e organizar melhor a lógica de inicialização do sistema, promovendo flexibilidade e escalabilidade.

Os Padrões Estruturais abordam a forma como classes e objetos são organizados e conectados para compor sistemas mais complexos. No contexto de Machine Learning, esses padrões auxiliam na integração de diferentes bibliotecas, ferramentas e módulos, muitas vezes desenvolvidos de forma independente. Ao encapsular complexidades internas e definir estruturas mais claras, os padrões estruturais contribuem para a manutenção do código, a reutilização de componentes e a clareza dos pipelines, reduzindo a dependência entre partes do sistema.

Já os Padrões Comportamentais estão relacionados à definição de fluxos de execução, regras de decisão e comunicação entre objetos. Em projetos de ML, esses padrões são fundamentais para lidar com comportamentos dinâmicos, como a escolha de estratégias de validação, a reação a eventos de treinamento ou a adaptação a mudanças de contexto. A aplicação de padrões comportamentais permite variar

comportamentos sem comprometer a estrutura do sistema, tornando as soluções mais flexíveis e fáceis de evoluir.

Outro aspecto central abordado refere-se à colaboração em projetos de maior escala. Em equipes multidisciplinares, compostas por indivíduos cientistas de dados, engenheiros de ML e engenheiros de software, os Design Patterns funcionam como uma linguagem comum de projeto. Essa padronização facilita a leitura, a revisão e a manutenção do código, permitindo que diferentes profissionais compreendam rapidamente a estrutura do sistema e contribuam de forma eficiente para sua evolução.

Em síntese, os Design Patterns representam um elemento essencial para transformar soluções de Machine Learning em sistemas profissionais e duradouros. Ao estruturar corretamente a criação de objetos, a organização das estruturas e o controle dos comportamentos, esses padrões permitem enfrentar de forma eficaz desafios recorrentes de escalabilidade, modularidade e manutenção. A compreensão e aplicação desses conceitos constituem um passo fundamental para o desenvolvimento de soluções de ML alinhadas às exigências técnicas e organizacionais do mercado atual.

REFERÊNCIAS

AMEISEN, E. **Building Machine Learning Powered Applications**. Sebastopol: O'Reilly Media, 2020.

BURKOV, A. **Machine Learning Engineering**. [S. l.]: True Positive Inc., 2020.

FOWLER, M. **Refactoring: improving the design of existing code**. 2. ed. Boston: Addison-Wesley, 2018.

FREEMAN, E.; FREEMAN, E.; SIERRA, K.; BATES, B. **Head First Design Patterns**. Sebastopol: O'Reilly Media, 2004.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: elements of reusable object-oriented software**. Boston: Addison-Wesley, 1994.

HUYEN, C. **Designing Machine Learning Systems**. Sebastopol: O'Reilly Media, 2022.

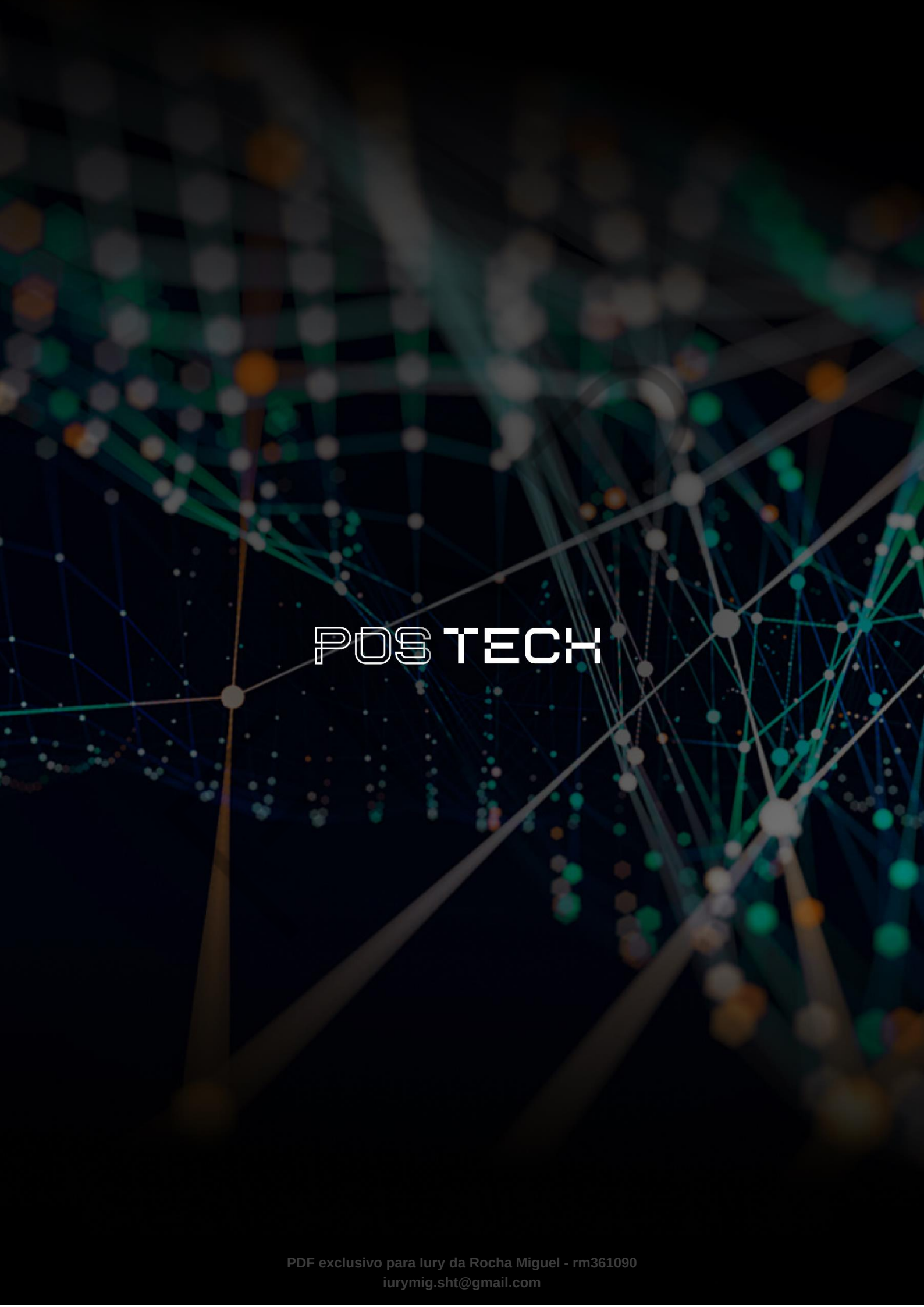
RAMALHO, L. **Fluent Python**. 2. ed. Sebastopol: O'Reilly Media, 2022.

SLATKIN, B. **Python design patterns**. [S. l.]: [s. n.], [s. d.].

PALAVRAS-CHAVE

Escalabilidade. Modularidade. Manutenibilidade.

EMSE



POSTECH